

Perspectives on refactoring planning and practice: an empirical study

Jie Chen^{1,2} · Junchao Xiao^{1,3} · Qing Wang^{1,3} ·
Leon J. Osterweil⁴ · Mingshu Li^{1,3}

Published online: 30 June 2015

© Springer Science+Business Media New York 2015

Abstract Iterative development increasingly seeks to incorporate design modification and continuous refactoring in order to maintain code quality even in highly dynamic environments. However, there does not appear to be consensus on how to do this, especially because research results seem to be inconsistent. This paper presents an empirical study based upon an industry survey of refactoring practices and attitudes. The study explored differences in attitudes about refactoring among participants who played roles in software development, and how these different attitudes affected actual practice. The study found strong agreement among all roles about the importance of refactoring, and agreement about the negative effects upon agility of deferring refactoring. Nevertheless, the survey found that roles had different perspectives on the different kinds of tasks in an agile process. Accordingly, there was no universally agreed-upon strategy for how to plan to carry out refactoring. Analysis of the survey results has raised many interesting questions suggesting the need for a considerable amount of future research.

Communicated by: Arie van Deursen

✉ Jie Chen
chenjie@itechs.iscas.ac.cn

Junchao Xiao
xiaojunchao@itechs.iscas.ac.cn

Qing Wang
wq@itechs.iscas.ac.cn

Leon J. Osterweil
ljo@cs.umass.edu

Mingshu Li
mingshu@iscas.ac.cn

¹ Laboratory for Internet Software Technologies, Institute of Software, Chinese Academy of Sciences, Beijing, China

² University of Chinese Academy of Sciences, Beijing, China

³ State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing, China

⁴ Department of Computer Science, University of Massachusetts, Amherst, MA, USA

Keywords Refactoring · Project Management · Software Process · Iterative Development · Survey

1 Introduction

Software undergoes continuous change resulting from various maintenance and evolution activities such as the addition of new functionality, correction of bugs, and adaptation to new environments. This is seen most clearly in iterative development approaches, which are well-established in software engineering as key ways to support the rapid change needed by quick-to-market applications. Agile software development processes (ASDs) are iterative development approaches that advocate principles such as the frequent delivery of partially implemented working systems, as opposed to the final delivery of only a completed product at the end of a long product development cycle (Alshayeb and Li 2005).

However, the continual reworking of partial implementations often causes software to become more complex and brittle, and to deviate from its original design, lowering the quality of the software, and sometimes making it harder to adapt the software as needed in response to market forces (Elssamadisy and Schalliol 2002; Cao et al. 2010; Belady and Lehman 1976). It has long been known that techniques such as code refactoring can be used to improve quality and enhance project agility (Moser et al. 2008; Fowler 1999). Thus, iterative development typically also incorporates simple design modification and continuous refactoring as approaches to maintaining code quality even in highly dynamic environments. Refactoring, in particular, is key in adding flexibility and extensibility without introducing new functionality (Kolb et al. 2006). But refactoring can be problematic because *the costs of refactoring are immediate* (Kataoka et al. 2002), *and can be quite significant, but its benefits are typically long-term, and may not be immediately visible or tangible* (Leitch and Stroulia 2003; Nidhra et al. 2012; Kim et al. 2012). *On the other hand, other tasks in the process such as fixing bugs and adding new features deliver immediate value.* Because of this it seems important to explore how best to incorporate refactoring into iterative development.

For this work we have assumed that all iterative development approaches incorporate an iteration planning activity, where the objectives of each iteration are discussed and established (Schwaber and Beedle 2002). In this planning activity, the development team typically has to come up with an effective strategy for addressing multiple goals, such as: creating new features for delivery and fixing bugs, while also making code more extensible, flexible, etc. (Schwaber and Beedle 2002; Drury et al. 2012). It is inevitable, that some of those diverse goals may come into conflict with others due to limited time and resources. Typically, an organization's iteration planning process focuses on addressing the product backlog, the product owner's prioritization of features that seem most urgently needed in order to meet the needs of rapidly evolving markets (Dong et al. 2011; Schwaber and Beedle 2002). The iteration planning process also takes into account the need to fix bugs. These bugs often are not listed in the product backlog, but that does not limit their importance. The need to consider refactoring is also part of most iteration planning processes. However, how best to integrate refactoring into the incremental plans that comprise the iterative process is less obvious, especially when the need for a refactoring does not seem to be directly related to improving the performance of the product or adding new features.

We believe, moreover, that it is important to consider possible differences in attitudes among various project participants, because planning decisions are made in the context of

environments in which consensus among different attitudes and perspectives must be reached. Thus, for example, product managers may tend to place more emphasis on lost business value when feature implementation may be delayed by refactoring (Kataoka et al. 2002). Developers may focus more on how their productivity might be increased when refactoring increases the quality of their code thereby facilitating understanding, maintaining, and modifying it (Moser et al. 2008; Beck 2000). Testers, on the other hand, may focus on increased testing effort and cost arising from the need for increased regression testing due to greater disruptions to code structure resulting from refactoring (Rachatasumrit and Kim 2012).

Indeed, there is conflicting evidence about the effectiveness of various strategies for planning and performing refactoring, as will be shown in the following two subsections:

For example, business stakeholders may have concerns about “big bang” software development and may therefore advocate scheduling numerous bug-fixing tasks aimed at delivering incremental product releases. But mandating tight scheduling constraints, may put off or prevent major redesigns, thus also putting off or preventing the creation of a reliable, stable, product with the flexibility needed to respond to rapid market changes (Kataoka et al. 2002; Bellomo et al. 2013). All too often, when decision makers pursue quick responses to the market, needed refactoring is deferred, code quality is reduced, and the refactoring that is eventually needed becomes more difficult and costly (Elssamadisy and Schalliol 2002). This means that, instead of undertaking proactive refactoring aimed at preserving code flexibility and quality, development teams often just wait for a business-driven opportunity to work on refactoring, or simply to wait until the performance of the product drops to a point that is unacceptable to the business side of the company. The emphasis of this approach is to focus on near-term goals, while still attempting not to ignore longer-term issues. Thus, for example, the product owner may wish to focus on the short-term goal of increasing the software product’s speed, while hoping that this focus does not significantly impede code flexibility or degrade code quality (Schwaber and Sutherland 2011).

Some processes such as the ones using the eXtreme Programming (XP) approach, on the other hand, advocates refactoring throughout the entire project life cycle, interspersed with other routine development activities, such as adding new functionality and fixing bugs (Drury et al. 2012). The advocates of this approach claim that continually putting off refactoring leads to code decay that can eventually cause the cost of adding the next handful of features to be roughly the same as the cost of rewriting the system from scratch. This is an expensive and dangerous outcome for any codebase that needs to retain significant business value. The XP approach suggests that refactoring the codebase continuously will result in making small improvements on a regular basis. But the primary advantage claimed for this approach is that it is effective in reducing the probability that massive re-design of the code will be needed. There is, however, some research that shows that this interspersed refactoring may often be ineffective and error-prone (Negara et al. 2012), especially when it is undertaken without sufficient time and resources to run the regression tests that are needed to ensure the correctness of refactoring edits (Rachatasumrit and Kim 2012).

In general, all of these refactoring approaches share the problem that their costs are immediate, and can be quite significant (Kataoka et al. 2002), but their benefits are often hard to see, except perhaps in the long run (Leitch and Stroulia 2003; Nidhra et al. 2012; Kim et al. 2012). Because of this, it can be difficult for decision makers to justify the (potentially high) immediate cost of refactoring by the difficult-to-quantify potential cost savings from a proposed refactoring. More specifically, it is hard for a decision maker to know, with confidence, by how much the quality of a project declines without refactoring, and how much

more it will cost to maintain and extend project quality through refactoring. Some previous work has been aimed at supporting the identification of so-called bad smells that might be used to suggest when to consider refactoring as a quality enhancement method (Elssamadisy and Schalliol 2002). And there are several theoretical and practical results that could be useful to decision makers attempting to do cost-benefits analysis of refactoring (Harman and Tratt 2007). But this existing work does not seem to us take sufficiently strongly into account the need to consider the ways that refactoring is regarded by the people playing different roles in an agile process and the ways that these roles compare the importance of refactoring to other kinds of process tasks.

In order to explore these issues, we have collected data through industry surveys. The project described in this paper undertook surveys aimed at gaining better understanding of refactoring planning and execution practice in agile processes. In this work we identified agreements and disagreements about various statements concerning the planning and performing of refactoring, and found differences between perceptions about current and preferred practices. We believe that these findings should provide valuable guidance towards topics for future research and practice.

Specifically, we have decomposed our main research questions as follows to study the survey results:

- **RQ 1:** Planning strategies. How does a development team plan for each kind of task during the planning for a development iteration?
- **RQ 2:** Perspectives on tasks. In what ways do people playing different roles in software development have different perspectives on the different kinds of tasks in an agile process?
 - **RQ 2.1:** Perspectives on workload for each kind of task. Do people playing different roles have different understandings of the workloads required to perform each kind of task?
 - **RQ 2.2:** Perspectives on planning for each kind of task. How do people playing different roles participate in planning decisions and how do they assign priorities to each kind of task during iteration planning?
- **RQ 3:** Different preferences. In what ways do people playing different roles in software development have different preferences for how priorities are assigned to different kinds of tasks in an agile process?
 - **RQ 3.1:** Satisfaction about priorities. Are team members generally satisfied with the priorities that are established in such processes?
 - **RQ 3.2:** Differences in preferences. Do the people playing different roles have different preferences for how the priorities are established in such processes?
- **RQ 4:** Impact of floss refactoring. How does floss refactoring affect the process?
- **RQ 5:** Practice of refactoring. How is refactoring actually practiced in the process?
 - **RQ 5.1:** Awareness of refactoring. How do different members of the development team actually become aware of the need for refactoring during the development process?
 - **RQ 5.2:** Planning and execution of refactoring. How are different levels of refactoring actually executed and planned during development?

Further, to obtain additional information that was not captured by the survey, we initiated discussions and conducted interviews that were aimed at learning whether there are generalizable rules for refactoring practice.

This paper is an extension of our previous work (Chen et al. 2014), which reached some general conclusions based on parts of the questionnaire. This paper includes the additional analysis of new research question (satisfaction of priority and awareness of refactoring) and further compares how attitudes vary among people playing different roles.

The rest of the paper is organized as follows: in section 2, we review related work. In section 3, we describe our research method. In section 4, we summarize the statistical analyses of our survey results. We discuss these results in section 5. And finally we present our overall research findings in section 6 and conclude the paper in section 7.

2 Backgrounds and Related Work

Iterative development is being seen increasingly widely as an effective approach to meeting the need for rapid response to market forces. Accordingly much research is focusing on how to plan the iterations that are a central feature of it. A key theme of much of this research seems to be maximizing the value of user stories (e.g., Przepiora et al. 2012; Brasil et al. 2012; Dong et al. 2011). Recently, however, there has been a growing emphasis on also addressing issues of speed and stability in ASD processes as well. Buchmann (Buchmann et al. 2012) observes that today's organizations must strive for both agility and sound design. Three tactics are presented to support reasoning about architectural tradeoffs between agility and stability. Dybå (Dybå and Dingsøyr 2008) found that lack of focus on architecture in ASD could lead to suboptimal design decisions. Similarly, losing sight of the big picture is also on the list of the top six concerns with agile development (Begel and Nagappan 2007). Recently, Bellomo (Bellomo et al. 2013) explored how agile projects deal with the pressure to rapidly deliver high-value capabilities and found that incorporating refactoring with other development practices can be useful in being able to respond quickly to unanticipated stability problems.

From the perspective of testing cost, it has been noted that refactoring always comes with a risk that it has introduced subtle bugs whose reliable discovery can require full regression testing. Ratzinger et al. (Ratzinger et al. 2008) found that as the number of refactoring edits increases the number of defects decreases. On the other hand, it has been found that a high ratio of refactoring edits is often followed by an increasing ratio of bug reports (Xing and Stroulia 2005; Zhang et al. 2011) and that incomplete or incorrect refactoring may cause bugs (Jones et al. 2002; Kolb et al. 2006). Further, Reddy describes a refactoring ripple effect using dependency oriented complexity metrics (Reddy and Rao 2009). Despite all of this, there is evidence that full regression testing usually not be sufficient to ensure the correctness of refactoring edits (Rachatasumrit and Kim 2012), and that inadequate regression testing can itself prevent developers from being free to undertake refactoring (Kim et al. 2012). To validating refactoring edits, SafeRefactor focuses on leveraging an existing test generation engine and by comparing test prior results to new program versions (Soares et al. 2010).

From the perspective of development improvement, Moser indicates that refactoring not only increases many dimensions of software quality, but also improves productivity (Moser et al. 2008). A number of other studies have addressed the problem of identifying where refactoring should be undertaken, for example, Sahraoui et al. used software measures to detect situations where a particular refactoring can be applied to improve the quality of a software

system (Sahraoui et al. 2000), Simon provides a distance based metric to identify where to apply which kind of refactoring (Simon et al. 2001). Marinescu provides a list of metrics-based detection strategies for capturing object oriented design flaws that suggest the need to refactor (Marinescu 2005). Considering refactoring in ASD, Chen et al. summarized the characterization of contexts in which a satisfactory architecture is likely to emerge from continuous small refactoring (Chen et al. 2014). In industry, there are many development tools such as Eclipse and Together Control Center can be used to automate software refactoring.

There have also been a number of empirical studies of refactoring. Moser (Moser et al. 2006) conducted a case study of the use of ASD in an industrial setting and found that refactoring enhances quality and reusability related metrics. Xing and Stroulia found that 70 % of structural changes in Eclipse's evolution history were due to refactoring (Xing and Stroulia 2005). Dig et al. studied the role of refactoring in API evolution, and found that 80 % of the changes that broke client applications were due to API-level refactoring (Dig and Johnson 2005). For more detail, Negara applied a continuous change analysis algorithm to the code evolution data from 23 developers working in their natural environment for 1520 h, and found that up to 42 % of some kinds of refactoring activities were not recorded as such in the Version Control System (Negara et al. 2012). Murphy-Hill et al. found that developers often interleave refactoring edits with other behavior-modifying edits and apply refactoring manually without using automated refactoring engines (Murphy-Hill et al. 2012).

Some work has also been done to try to understand attitudes about refactoring. A survey of 85 software developers was done to study their awareness of code smells (Yamashita and Moonen 2013). Another study was done to study the attitudes of Microsoft developers towards the concepts behind refactoring concept and the benefits of refactoring (Kim et al. 2012). Also, an interview done by Chen et al. has found that different people have different views about whether or not a satisfactory architecture emerges through continuous refactoring (Chen and Babar 2014).

None of these papers, however, considers sufficiently carefully the different ways in which refactoring is viewed by the people playing different roles in agile processes and how planning and performance of refactoring should be balanced with other development tasks.

3 Research Method

The basis for our study of how refactoring is planned and practiced in industry was a questionnaire-based survey. Broadly, our research was carried out in three phases: Phase 1: Designing the survey, Phase 2: Administering the survey, Phase 3: Analyzing the survey data. We now describe each phase in more detail.

3.1 Designing the Survey

As a first step, we performed a small-scale pilot study to solidify our understandings of the central research issues and to define the goals of the survey. The survey was designed to focus on:

- **Planning practice:** Exploring each of the different types of tasks considered during iteration planning, and determining if the people playing different project roles have different views of those types of tasks.

- **Refactoring practice:** Exploring how refactoring is currently practiced within industry, how projects are impacted by deferred refactoring, and how the people playing different project roles differ in their views about refactoring practice.

The questionnaire comprised four parts:

- **Background information.** We gathered information about the respondents, including both personal information and information about the agile team to which the respondent belonged.
- **ASD team planning practices.** We presented closed-ended questions (Fink 2003) asking respondents to describe: 1) the percentage of workload a developer spends on each kind of task in a typical iteration, 2) the roles that different people play in iteration planning, and the authority that each role has in the planning, 3) the priority that each kind of task is given in iteration planning, and 4) the strategy used when planning each kind of task.
- **ASD team Refactoring practices.** We presented questions aimed at determining the levels of agreement about various experiences in performing refactoring. Answers were given on a Likert Scale ranging from 1 to 5, namely: (1) strongly disagree, (2) disagree, (3) neither agree nor disagree, (4) agree, (5) strongly agree.
- **Personal opinions.** We also presented open-ended (Fink 2003) questions to allow respondents to provide personal opinions and additional information, insights, and perspectives not captured by the closed-ended questions.

The questionnaire was pre-tested for validity to identify and remove ambiguities, improve identified readability problems, and establish an effective ordering of the questions. Pre-testing was done by ten well-qualified software development professionals who had different experience levels and work functions and worked in both local and global corporations in China.

3.2 Administering the Survey

We collected data by distributing the questionnaire online to software developers at different IT companies. We recruited our participants by distributing our survey with an email invitation and by making it available to developers through online social networks such as LinkedIn. Special interest groups within those networks, especially those that deal with software engineering in general, and agile development in particular, provide platforms for the discussion of relevant topics by professionals, and so seemed to be logical places in which to seek particularly informed responses to our questionnaire. We did not restrict our surveying to any particular industry type, sector, or size. The only requirement we had was that the respondent had practiced ASD. More specifically, we required that, 1) the respondent had been a part of a team that had adopted ASD practices (to do this we eliminated the respondents with no ASD experience), and 2) the respondent had good understandings of software development approaches in general, and refactoring in particular.

The survey was administered using a web-based questionnaire, which can be found at <http://www.wenjuan.com/s/eA3Qn2/> (as shown in the Appendix). We received 126 responses, of which 105 were eligible responses as summarized as percentages in Table 1. The respondents came from all over the world, suggesting good international coverage, but the majority of the respondents came from either China (35.2 %) or North America (30.5 %).

Table 1 Characteristics of the survey sample

Individual		Team	
Experience		Team Size	
<1 year	12.40 %	1–5	26.70 %
1–3 years	20.00 %	6–10	36.20 %
3–5 years	20.00 %	11–20	16.20 %
>5 years	47.60 %	21–40	6.70 %
		>40	12.40 %
Experience in Agile		Product Owner	
<1 year	31.40 %	Customer	26.30 %
1–3 years	27.60 %	Team	51.50 %
3–5 years	24.80 %	Others	21.20 %
>5 years	16.20 %		
Role		Location	
Project Manager	18.10 %	China	35.20 %
Product Manager	7.60 %	North America	30.50 %
Architect	20.00 %	South America	1.00 %
Development Engineer	50.50 %	Europe	14.30 %
Test Engineer	21.00 %	Oceania	2.90 %
UED/Operator	5.80 %	Africa	1.90 %
Expert	9.50 %	Asia (besides China)	14.30 %
Organization Size		Iteration Length	
<50	13.30 %	One week	17.10 %
50–200	15.20 %	2–4 weeks	59.00 %
200–500	1.90 %	1–3 months	14.30 %
500–1000	7.60 %	3–6 months	1.90 %
>1000	61.90 %	6–12 months	1.90 %
		More than one year	1.00 %
		Uncertain	4.80 %

Approximately 61.9 % of the respondents came from large companies (1000+ employees), which we expected to be well organized and to do a good job of process management. Most of the respondents (about 62.90 %) belonged to teams having 1–10 employees, and 76.1 % of the respondents were working on projects that were decomposed into short iterations (shorter than one month), making them suitable for the application of agile development principles. As indicated in Table 1, more than half of the products were owned by the team instead of the by customers or company employees who were not team members.

We allowed respondents to indicate that they had more than one role, but categorized the respondents by the role that they claimed as their main responsibility in their project. We were particularly interested in acquiring more detailed information about developers as they were the main population of our survey and they were responsible for creating products and executing iteration planning. Approximately 39.6 % of developers were relatively more experienced (5+ years) and 67.9 % of developers had been working in an agile development

context for more than one year. We also invited participation from respondents whose roles were related to decision making during iteration planning, being particularly interested in comparing their opinions to those of the developers. These roles included project manager (responsible for ensuring the successful completion of work-in-progress), product manager (representing customers' interests), architect (setting the technical direction of the overall project), tester (responsible for preventing defects from entering the system), UED (responsible for assuring system usability), operator (responsible for keeping products running during production), and agile expert (assuring correct use of agility approaches by serving as scrum master/agile coach). All of the agile experts and operators considered themselves experienced, and most of the project managers (78.9 %) and architects (71.4 %) also considered themselves to be experienced. But only 33.3 % of the UED, 40 % of the testers and 50 % of the product managers considered themselves experienced.

3.3 Analyzing the Data

Our data analysis approach was both quantitative (involving statistical analysis of the closed-ended questions in the questionnaire) and qualitative (studying and interpreting replies to the open-ended questions). We performed different statistical studies of the answers to the closed-ended questions.

First we generated descriptive statistics to help summarize the information latent in the data that we obtained from the survey questions. These initial relatively simple analyses produced percentage distributions, minima, maxima, means, and standard deviations of the survey data obtained from respondents.

But, going more deeply, we wanted to study such more subtle questions as how strongly a person's background affects the perceptions of and satisfaction with factors related to refactoring in the processes. On the basis of the survey data we obtained, respondents were categorized according to factors such as whether or not the respondent was a *Developer*, and whether or not the respondent was *Experienced* (determined by whether or not the respondent had worked for more than 5 years). As shown in Table 1, there was good response from both *Developer* (50.5 %) and *Non-Developer*, *Experienced* (47.6 %) and *Less-Experienced* respondents.

For our statistical assessments, we mainly applied the Chi-Square Test to evaluate hypotheses such as whether *proportions of the selections made by one group match the proportions of selections made by another group*. The Chi-Square test helped us to detect significant differences in underlying perceptions between each of two different groups. For all the test statistics presented in this paper, the significance level we used was set at $p=0.05$ or $p=0.01$. We performed these tests using various packages in the R statistics package (Venables et al. 2002).

4 Survey Results

In the following subsections, we analyze how the responses that we received shed light upon our stated research questions.

4.1 Planning Strategies

In this section, we address the question: **How does a development team plan for each kind of task during the planning for a development iteration?**

In an attempt to understand various planning strategies, the questionnaire asked whether each of the following four kinds of development process tasks is considered during iteration planning:

- **NF:** tasks aimed at creating new features
- **BF:** tasks aimed at fixing bugs
- **FR:** tasks to support floss refactoring
- **RR:** tasks to support root-canal refactoring

The last two kinds of refactoring were so-named previously in (Murphy-Hill et al. 2012), and explained as follows. In floss refactoring, the programmer intersperses refactoring with other kinds of program changes to keep code healthy. Root-canal refactoring, in contrast, is used to correct deteriorated code and is a protracted process consisting exclusively of refactoring.

We also asked which of the following planning strategies were used:

- **S1.** Do not make a plan to accomplish a task, but leave a needed task to be done when someone has spare time;
- **S2.** Do not make a plan to accomplish a task, but leave some slack time to allow for the task to be done;
- **S3.** Plan that a needed task will be done as part of another task;
- **S4.** Make a simple plan for the critical tasks;
- **S5.** Make a specific detailed plan for all tasks.

Two more answer options were also provided:

- **S6.** I don't know,
- **S7.** Others.

Table 2 summarizes the results as percentages of responses. Multiple-selection was allowed for this question.

Our survey showed that a large proportion (around 98.1 %) of the respondents said that tasks relating to implementing new features are carefully planned (the respondents selected either S4 or S5). And indeed none of the respondents selected the “I don't know” choice for NF. Among the 1.9 % who selected strategy 1 for NF, one was from an organization with less 50 employees and the respondent indicated that they just do the best they can. Others selected

Table 2 Planning strategies for each kind of task

	S1	S2	S3	S4	S5	S6	S7	Total
RR	16.20 %	16.20 %	25.00 %	17.60 %	25.00 %	11.80 %	1.50 %	113.30 %
FR	21.90 %	19.00 %	13.30 %	27.60 %	16.20 %	7.60 %	0.00 %	105.60 %
BF	6.70 %	21.90 %	11.40 %	45.70 %	22.90 %	1.00 %	0.00 %	109.60 %
NF	1.90 %	2.90 %	0.00 %	44.80 %	53.30 %	0.00 %	0.00 %	102.90 %

other strategies in addition to S1, suggesting that S1 is chosen depending upon the nature of the feature.

Similarly, the great majority (about 68.6 %) of respondents indicated that bug fixing was also carefully planned (the respondents selected either S4 or S5), further reinforcing the view that there is uniformly careful attention paid to planning for these two types of tasks, and corroborating conventional wisdom that creating new features and fixing bugs are regarded as the most important development tasks.

The responses addressing the type of planning done for both kinds of refactoring were far more evenly divided. About half of respondents claimed that they had applied careful planning for either root-canal (42.6 %) or floss refactoring (43.8 %) tasks. Further, for both kinds of refactoring, about similar proportions (about 40 %) of the respondents claimed that they do not do careful planning at all for both kinds of refactoring (they selected either S1 or S2) and about 10 % of the respondents said that they had no idea. In addition, two respondents selected “Other strategy” and one respondent noted that they do root-canal refactoring only upon a request from either the customer or top company management. Another respondent observed that the team does not plan for root-canal refactoring as a separate task but deals with it as a new feature aimed at increasing customer satisfaction with product performance. The lack of focus on root canal refactoring was further reinforced by noting that the total percentage of approaches to doing RR tasks aggregated 113.2 % (recall that multiple responses were allowed), indicating a relative lack of consensus on how to address these kinds of tasks.

Summary: Agile teams pay close attention during iteration planning to tasks involved in creating new features and bug fixing, but do not seem to have a unified view of how to approach planning for refactoring.

4.2 Different Perspectives

This section focus on addressing the question: **In what ways do people playing different roles in software development have different perspectives on the different kinds of tasks in an agile process?**

4.2.1 Perspectives on Workload

First, we analyzed our data to answer: **Do people playing different roles have different understandings of the workloads required to perform each kind of task?**

Table 3 summarizes the responses to our questions aimed at investigating how a respondent’s role impacts their view of the workload perception required to complete each type of tasks for developers in their team. Each cell in Table 3 represents the percentage of total responses, for each respondent type and each of the task types specified on the left of each row, that fell in the range specified at the top of the column. More specifically, for this study, we divided the respondents into two groups: *Developer* (50.5 %) and *Non-Developer* (49.5 %), and respondents from both groups responded to the question “*what proportion of workload does a development engineer spend for each type of task*”. For each type of task, we provided five possible answers: <1 %, 1–10 %, 11–30 %, 31–50 %, >50 %. For each type of task, we applied the Chi-Square Test to examine the hypothesis that:

Table 3 Workload percentage for each kind of tasks

		<1 %	1 %–10 %	11 %–30 %	31 %–50 %	>50 %	I don't know	Sig.
NF	Non-Developer	0.0 %	5.8 %	25.0 %	23.1 %	46.2 %	0.0 %	0.001**
	Developer	0.0 %	13.2 %	9.4 %	54.7 %	22.6 %	0.0 %	
BF	Non-Developer	3.8 %	17.3 %	46.2 %	26.9 %	5.8 %	0.0 %	0.209
	Developer	0.0 %	34.0 %	35.8 %	22.6 %	7.5 %	0.0 %	
FR	Non-Developer	15.4 %	36.5 %	36.5 %	3.8 %	1.9 %	5.8 %	0.265
	Developer	18.9 %	35.8 %	22.6 %	17.0 %	1.9 %	3.8 %	
RR	Non-Developer	13.5 %	48.6 %	27.0 %	2.7 %	0.0 %	8.1 %	0.437
	Developer	25.0 %	44.4 %	19.4 %	8.3 %	0.0 %	2.8 %	

**Correlation is significant at the 0.01 level (2-tailed)

*Correlation is significant at the 0.05 level (2-tailed)

proportions of the selections made by the Non-Developer match the proportions of selections made by the Developer.

We observed that there was no significant difference between the answers provided by the members of the two groups except in the case of tasks involved in creating new features, where there was a significant difference between two groups (with $p=0.001<0.01$) as shown in Table 3. Thus the statistical result rejects the hypothesis significantly for tasks involved in creating new features.

Further, we found that the significant difference arises from a difference between the two groups of respondents in their selections of 30 %–50 % and >50 %. Amongst the respondents belonging to the group of *Developer*, the percent of workload for tasks involved in creating new features was mostly concentrated (54.7 %) in the 30 %–50 % range. However, only a small number of *Non-Developer* (23.1 %) made the same selection, with most (46.2 %) selecting the workload as being >50 %. This suggests that developers actually spend a significant amount of time doing work that is other than creating new features, but may do so in ways that seem to be unobservable or invisible to those having different roles.

Results obtained for other kinds of tasks (BF, FR, RR), provide supporting evidence for the above observation. For bug fixing tasks, 46.2 % of the *Non-Developer* responded that the workload is 11–30 % while the *Developer* selected the three choices: 1 %–10 %, 11 %–30 %, 31 %–50 % equally frequently. About 18.9 % of the *Developers* said that they spend more than 30 % of their workload on floss refactoring while only 5.7 % of the *Non-Developer* thought that was the level of workload entailed. There was a relative consensus about the workload for root-canal refactoring. Most respondents from both groups claimed 1 %–10 % of the workload. There was a small difference between the two groups in that more *Non-Developer* selected a workload of more than 10 % and more *Developer* selected a workload of less than 1 %. For both kinds of refactoring, more *Non-Developer* answered, “I don’t know”.

Summary: Most *Non-Developers* thought more than 50 % of the development workload is spent creating new features in each iteration, while most *Developer* thought only 31–50 % of their workload was spent creating new features. This suggests that developers spend more (unplanned) time and effort on tasks such as refactoring than is realized by *Non-Developer*.

4.2.2 Perspectives on Planning

Then, we addressed the question: **How do people playing different roles participate in planning decisions and how do they assign priorities to each kind of task during iteration planning?**

To answer this question, we sought to understand how the roles take part in making planning decisions and how do the roles assign priorities to each of the different kinds of tasks.

First, the questionnaire categorized related roles in iteration planning as shown in Table 4. An assessment of the authority of each role in iteration planning was rated on a scale of 0 to 5, where 0 meant that the role does not participate at all in planning decisions, while 5 means that the role has the ultimate authority for all decisions in iteration planning. On average, the product manager had the highest authority level among the four roles with a mean value of 3.65. This was followed closely by the project manager with a mean value of 3.50. The distribution of the answers about the authority of the project manager and product manager was negatively skewed meaning that most of the respondents agreed that these roles had high levels of authority (with median value of 4). At the opposite end, testers were considered to have the lowest authority level.

Then, for each of the different kinds of tasks, the respondents were asked to rate the priority used to be assigned in their team to each kind of planning task on a scale of 0 to 5 (where 0 means the task is not considered at all, and 5 means the task is given the highest priority). On average, NF and BF tasks were found to have relatively higher priorities. Further, the results revealed significant evidence of negative skewing and excess kurtosis for tasks involved in creating new features, indicating these tasks get the highest priority. In contrast, responses rating the priorities given to tasks involved in both kinds of refactoring were nearly normally distributed, with a mean value around 2.25, suggesting lack of clear consensus, but notably lower priority.

In addition, we used correlation analysis to help us understand the relationship between role authority in planning and the priority for each kind of task. We used correlation analysis to help us understand which relationships were positively correlated, which were negatively correlated, and the strength of these relationships. We computed two sets of Spearmans's coefficients of correlation and significance values at levels 0.05 and 0.01. Table 5 shows the summarized correlation results for items with non-missing values.

Table 4 Summarized descriptive statistics

Role	Label	Mean	Median	Std. Dev	Skewness	Kurtosis
Project Manager	R1	3.50	4	1.527	−0.835	−0.307
Product Manager	R2	3.65	4	1.532	−1.120	0.331
Developer Engineer	R3	3.45	3	1.083	−0.418	0.061
Test Engineer	R4	2.49	3	1.370	−0.191	−0.565
Task	Label	Mean		Std. Dev	Skewness	Kurtosis
Creating New Feature	NF	4.10	4	0.950	−1.164	1.245
Bug Fixing	BF	3.80	4	1.152	−0.642	−0.339
Floss Refactoring	FR	2.24	2	1.179	−0.124	−0.113
Root-canal Refactoring	RR	2.26	2	1.390	−0.124	−0.797

Table 5 Summarized Spearman correlations table

	R1	R2	R3	R4	NF	BF	FR	RR
R1	1							
R2	0.246*	1						
R3	-0.073	0.093	1					
R4	0.213*	0.072	0.269**	1				
NF	0.082	0.283*	0.132	0.173	1			
BF	0.315**	0.134	0.065	0.217*	0.023	1		
FR	0.050	0.030	0.140	0.100	-0.123	0.201*	1	
RR	0.093	0.128	0.071	0.013	-0.235*	0.136	0.589**	1

The data shows a significantly positive correlation between the authority in iteration planning of developers and testers, meaning that the authority of the developers and testers rises or falls pretty much in unison within a single iteration planning. Similarly, the authority of the project manager is positively correlated with both the product manager and testers. We also found a significantly positive correlation between the priorities of the two kinds of refactoring tasks, and also between bug fixing priority and floss refactoring priority. However, we found a negative correlation between the tasks involved in creating new features and tasks involved in both kinds of refactoring, and significant negative correlation between NF tasks and RR tasks. These results corroborated our prior intuition that when a team is focused on adding new features, refactoring tasks are reduced to low priority.

It is inevitable that people having different roles will have different responsibilities and thus that their goals may be different. Accordingly, we wanted to explore if people having different roles prefer to see different kinds of tasks pursued. It is easy to see that the product manager's authority was statistically significantly correlated with the priority of NF in iteration planning. This supports the hypothesis that when the product manager has higher decision-making authority the priority for NF is correspondingly higher. Similarly, testers and project managers placed a higher priority on bug fixing, which also reflects their responsibilities.

We also found that developers' authority is not significantly correlated with the priority for any particular kind of task. Although we could not find a significant effect of the decision makers' role on the priority of either kind of refactoring, we did notice that the roles of developer and tester tended to be much more concerned with FR. In contrast, the product manager role tended to be much more concerned with RR, which is used for restructuring deteriorated code and involves a considerable amount of work over a protracted period of time.

Summary: There is significant evidence that different roles have distinctly different preferences for the different kinds of tasks during iteration planning. Product managers give higher priority to creating new features, while project managers and testers give higher priority to bug fixing.

4.3 Different Preferences

The way in which priorities are assigned affect the way in which refactoring is practiced. Thus we analyzed how priorities are assigned to each kind of task to address the question: **In what**

ways do people playing different roles in software development have different preferences for how priorities are assigned to different kinds of tasks in an agile process?

4.3.1 Satisfaction of Priority

Generally, we analyzed the question: **Are team members generally satisfied with the priorities that are established in such processes?**

To study how satisfactory the planning process is to the various types of team members, we compared the priorities that team members wanted to see assigned to the various types of activities to the priorities actually assigned. Specifically, the survey asked respondents the priorities that they would like to see assigned to each of the different kinds of tasks, where priority was measured on a scale from 0 to 5, and the priority that they actually be assigned (as analyzed in section 4.2.2). Thus, for each kind of task, we compared a pair of priorities, the expected priority, and the priority actually set by the team. We then computed team member satisfaction as a measure of the difference between the two. In this study we defined individual satisfaction as the difference between the two values. Hence, a large difference indicated a high degree of dissatisfaction with the current priority. A positive value for the difference indicated that the team member would like to see the current priority increased, while a negative value meant that too much emphasis was currently being placed upon it.

We averaged and normalized the results to facilitate comparison in Table 6. As can be seen from this table, more than half of the respondents seemed to feel that creating new features and fixing bugs were given the appropriate priority (the difference between the pair of priorities was nearly 0). More specifically the -0.049 mean difference, for the NF type of task suggests that there is almost an equal desire to increase its priority as to decrease its priority. On the other hand, however, there is a huge range in satisfaction levels, with some respondents wanting to reduce the priority for this task by as much as 4 points with other respondents wanting to increase its priority by as much as 2 points. The situation for BF tasks was similar, with more than half of the respondents feeling that this task receives the appropriate priority level, but with significant numbers of respondents also showing widely ranging (from -4 to $+4$) dissatisfaction with the priority given to this kind of task.

On the other hand, the mean value of differences for both kinds of refactoring showed that respondents would like to increase the priority of this kind of task, with about half expecting that a higher priority should be assigned to this kind of task than was actually assigned. On average, the desired increase in priority for refactoring was 0.565 and 0.812 for RR and FR respectively. And, indeed, the largest measured difference between expected and observed

Table 6 Summarized priority difference

Task	Expected – real priority (Valid percent)			Descriptive statistics				Sig.
	<0	=0	>0	Min	Max	Mean	Std. Dev	
NF	22.6 %	55.9 %	22.5 %	−4.00	2.00	−.049	1.084	0.790
BF	16.6 %	55.3 %	28.2 %	−4.00	4.00	.185	1.135	0.114
FR	14.9 %	33.7 %	51.4 %	−2.00	5.00	.812	1.419	0.000**
RR	20.9 %	29.0 %	49.9 %	−3.00	5.00	.565	1.595	0.010*

priorities, a full 5 points, corresponding to the largest value on the 5-point scale was noted in the responses both for FR and RR. This corresponds to a response in which the respondent wanted to assign the highest priority to refactoring but observed that the respondent's team was not planning on doing refactoring at all.

We then tried to determine whether the observed differences between desired and observed priorities are significant. We used the Wilcoxon signed-rank test, a non-parametric statistical hypothesis test used when comparing two related samples, matched samples, or repeated measurements on a single sample, to assess whether the observed differences was significant. In this study, the hypothesis we used was:

The median of differences between real priority and expected priority equals 0.

According to the Wilcoxon signed rank test, we found that the differences between real priorities and expected priorities are not significant for NF and BF at the 0.05 and 0.01 levels. This indicates that the priorities assigned to both of these kinds of tasks seemed acceptable. However, significant differences existed between the real priority and the desired priority for both kinds of refactoring at 0.05 level ($p=0.00$ for floss refactoring and $p=0.0105$ for root-canal refactoring).

Summary: There is significant evidence that the majority of the respondents were not satisfied with the priorities established by their team for both kinds of refactoring and would like to see then receive a higher priority.

4.3.2 Differences in Preferences

Then, considering the background of the respondents, we addressed the question: **Do the people playing different roles have different preferences for how the priorities are established in such processes?**

We carried out additional analyses to study the degree to which differences in respondents' responses about desired and observed priorities might be attributable to differences in their backgrounds. To do this we divided the respondents into two groups, *Experienced* and *Less-Experienced* respondents. Table 7 summarizes the results. Overall, we note that respondents from the *Experienced* group are more inclined than respondents from the *Less-Experienced* group to increase the priority given to all four kinds of tasks. Notably, *Less-Experienced* respondents would like to decrease the priority of NF and BF tasks, while increasing the priorities of the two refactoring types of tasks, although the desired increases in priorities for the two refactoring tasks were still lower than the increases desired by the members of the *Experienced* group.

Further analysis of the data using a significance test reveals that the differences between the responses gotten from the two groups of respondents for bug fixing (BF) type tasks is significant at the 0.05 level, with a p-value of 0.019. More than half of the respondents from each of the two groups stated that the priority given to this type of task was at the level that they desired. On the hand, however, the other half of the *Experienced* respondents expressed significant dissatisfaction with observed priorities, with the mean amount of dissatisfaction being 0.542. The other half of the *Less-Experienced* respondents on average wanted to see a decrease in the priority given to BF tasks, with their dissatisfaction averaging -0.127 . This shows a sharp difference in attitude, with the *Experienced* respondents showing far more interest in bug fixing.

Table 7 Summarized priority difference for Experienced and Non-Experienced respondents

Task	Experienced			Less-experienced								Sig.			
	<0	=0	>0	Min	Max	Mean	Std. Dev	<0	=0	>0	Min		Max	Mean	Std. Dev
NF	17.0 %	57.4 %	25.5 %	-2.00	2.00	0.127	0.899	27.3 %	52.7 %	20.0 %	-4.00	2.00	-0.200	1.208	0.443
BF	8.3 %	52.1 %	39.6 %	-2.00	4.00	0.542	1.202	23.6 %	58.2 %	18.2 %	-4.00	2.00	-0.127	0.982	0.019*
FR	19.1 %	25.5 %	55.3 %	-1.00	4.00	1.021	1.567	11.1 %	40.7 %	48.1 %	-2.00	5.00	0.630	1.263	0.215
RR	12.5 %	28.1 %	59.4 %	-3.00	4.00	0.906	1.532	30.0 %	30.0 %	40.0 %	-3.00	5.00	0.200	1.600	0.179

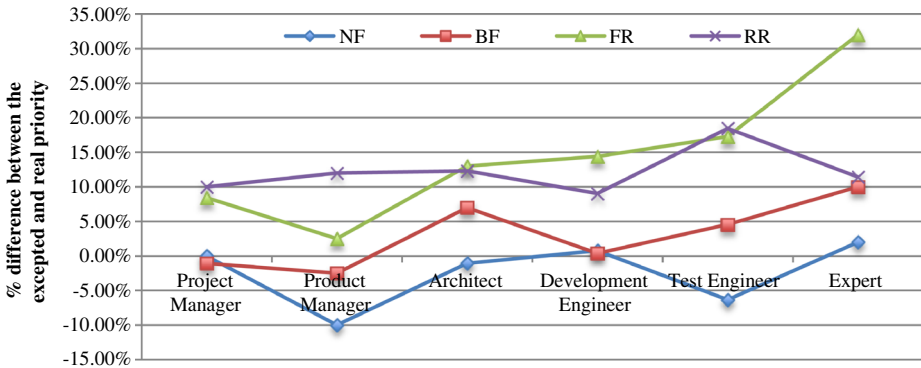


Fig. 1 Difference between expected and real priority

We then considered the effect of respondent roles on responses. Figure 1 shows how respondents with each role responded on average. The figure shows that all respondents wanted to attach a higher priority to both kinds of refactoring. Respondents wanted to see the priority of root-canal refactoring increased by more than 10 %. Product managers seemed content with the existing priority given to floss refactoring, while experts (scrum masters/agile coaches) wanted to see a substantial increase in this priority. It seems to make sense that, because floss refactoring is usually not required for implementation of a new feature, the product managers would see little value in it. Architects, testers and experts all wanted a higher priority given to bug fixing, but the other roles seemed to have little desire for any changes in priority. The most unexpected finding was that product managers seemed to want a lower priority placed on creating new features. This may be because product managers were as concerned about product quality as about meeting customers' demands for new features.

Summary: There is significant evidence that the majority of the respondents were not satisfied with the priorities that their teams assigned to both kinds of refactoring and would have liked to have seen a higher priority assigned to them.

4.4 Impact of Floss Refactoring

The previous analyses suggested that floss refactoring was typically performed as some kind of special task, namely one that did not add features or fix bugs. To explore this further, we asked the respondents to indicate how they thought floss refactoring impacts the overall development process, addressing: **How does floss refactoring affect the process?**

The questionnaire offered a choice of five different kinds of impacts, four negative impacts and one positive impact. For the negative impacts, the question was asked in the form of “in the project, deferred floss refactoring will ...”:

- **IMP 1.** Reduce productivity in adding new features in related components;
- **IMP 2.** Increase the workload of adding new features in related components;
- **IMP 3.** Increase the defect density of adding new features in related components;
- **IMP 4.** Require a lot of extra effort to clean up the code later.

The one positive impact was described as:

- **IMP 5.** Root-canal refactoring can be avoided by frequent floss refactoring.

Respondents were asked to indicate their responses on the 1 to 5 Likert scale (1 – Strongly Disagree, to 5 – Strongly Agree). The left part of Table 8 shows the descriptive statistical results. As expected, the results demonstrated a clear consensus, with most of the respondents (60 %–79.6 %) agreeing (by choosing either agree or strongly agree) about the impact of floss refactoring upon the process. We observed, moreover, that none of the replies had a mean score below 3 (“Neither Agree nor Disagree” on the 5-point Likert scale), which means that very few participants had a positive view of the impact of refactoring on the overall process. On the other hand, we noted that refactoring seems to be generally accepted, both in theory and in practice.

The middle column of Table 8, shows a comparison between the results obtained from both the *Developer* and *Non-Developer* groups. The comparison indicates that the *Developer* showed more agreement on the positive impact of refactoring with a high mean score of 4.04, while the *Non-Developer* group replies focused more on the four negative impacts and had a higher mean score. Using a Chi-Square Test, we found that the difference between the two groups was statistically significant, with a level of $p=0.05$ for the comparison of replies to the question about the impact of refactoring upon defect density. Our comparison of the responses from *Experienced* participants and *Less-Experienced*, participants showed no significant difference (as shown in the column on the right side of Table 8). These results showed that, in general, the respondents had high agreement about the negative impacts of refactoring. Although we could not find a significant impact of the respondents’ experience upon their opinion of the impact of refactoring, we did notice that the experienced respondents tended to be more concerned about the impact upon workload that is attributable to adding new features. The *Experienced* group showed the largest difference in increased mean score for this response.

Summary: Deferred floss refactoring is generally considered to have all four of the listed negative effects. This was somewhat supplementing in view of the commonly-held belief (especially in the eXtreme Programming community) that frequent floss refactoring is helpful in avoiding the need for later root-canal refactoring (Kataoka et al. 2002).

4.5 Practice of Refactoring

To explore this further, we asked the respondents to indicate how they practice refactoring in the development process, addressing the research question:

RQ 5: How is refactoring actually practiced in the process?

The respondents were asked to indicate their responses on the 1 to 5 Likert scale (1 – Strongly Disagree, to 5 – Strongly Agree) for each of the two statements labeled as RQ 5.1 and RQ 5.2, that we elaborate upon now.

4.5.1 Awareness of Refactoring

We began our study by investigating forward-looking attitudes about refactoring, addressing the question: **How do different members of the development team actually become aware of the need for refactoring during the development process?** Specifically we tried to

Table 8 Summarized agreement of the refactoring impact

	Descriptive statistics			Developer vs non-developer			Experienced vs less-experienced			
	Mean	Std. Dev	%Agree	%Disagree	Developer	Non-developer	p-value	Experienced	Less-experienced	
					p-value	p-value				
IMP1	3.53	1.089	60.0 %	19.6 %	3.42	3.69	0.269	3.47	3.64	0.596
IMP2	3.69	1.009	66.7 %	15.6 %	3.63	3.74	0.537	3.81	3.57	0.069
IMP3	3.70	1.056	67.0 %	16.0 %	3.46	3.96	0.017*	3.69	3.71	0.418
IMP4	4.05	.957	79.6 %	8.2 %	3.96	4.14	0.323	4.08	4.02	0.188
IMP5	3.85	.911	78.2 %	10.9 %	4.04	3.67	0.245	3.76	4.00	0.506

determine respondents' level of agreement about the statement "*when development engineers create new features, they consider extensibility and maintainability for most possible future changes*". A summary of the responses is shown in the left of Table 9. The mean score of all responses was 3.05, suggesting that, on average, the participants were moderately concerned about this statement. Specifically, 43 % of the development engineers disagreed (replied with either disagree or strongly disagree) with the statement and 35 % of the respondents agreed (replied with agree or strongly agree).

The respondents were also asked to evaluate the statement "*In the project, the development engineers usually identify the necessity of floss refactoring before it must be done (the product has poor performance/blocks new changes)*". The mean score for all responses was 3.31, indicating more concern about this statement. In particular, a majority of the respondents (57.0 %) agreed (replied with either agree or strongly agree) that development engineers usually identify the necessity of floss refactoring before it must be done. A much smaller percentage of respondents (30 %) did not agree with the statement (replied with either disagree or strongly disagree).

To better understand these results we undertook further analysis to try to understand how the impact of the background of the respondents may have affected these results. To do this we divided the respondents into two groups according to their background and analyzed their responses. The results are shown in the remaining parts of Table 9.

As both of the statements are directly related to the behavior of the developers, we compared the replies from the *Developer* and *Non-Developer* groups. No significant difference was found between these two respondent groups at the 0.05 level.

On the other hand, we considered how the experience of the respondents might impact the observed results. We did this by dividing the respondents into *Experienced* and *Less-Experienced* groups. Here we did find significant differences, especially in the level of concern about the awareness of the need for refactoring. We measured the difference at the 0.05 level where $p=0.048$. Thus, the *Experienced* group tended to agree much more that developers can identify the need for floss refactoring before it must be done, as evidenced by their mean score of 3.54. On the other hand, we found that the *Non-Experienced* respondents were less agreed (with the mean score 2.92 which is less than the neutral score of 3) that developers are likely to effectively anticipate upcoming needs for refactoring. However, the average score for the *Experienced* respondents was greater than 3, indicating that the experienced respondents showed more agreement than the developers about forward-looking refactoring.

Summary: Participants were moderately concerned about the loss of extensibility and maintainability that might result from the changes need to create a new feature. In addition, development engineers were generally trusted to identify the necessity of floss refactoring before it must be done. However, the experienced respondents were more in agreement about this than the Less-Experienced respondents.

4.5.2 Planning and Execution of Refactoring

In the next phase of our studies, we addressed the question: **How are different levels of refactoring actually executed and planned during development?** We analyzed how refactoring is actually planned and executed in the development process. Refactoring is carried out at a wide range of levels, from as low a level as single expressions to as high a level as whole inheritance hierarchies. For this survey we characterized three different levels as suggested by (Murphy-Hill et al. 2012):

Table 9 Summarized agreement of the refactoring practice

	Descriptive statistics			Developer vs non-developer			Experienced vs less-experienced			
	Mean	Std. Dev	%Agree	%Disagree	Developer vs non-developer		Experienced vs less-experienced			
					Developer	Non-Developer	Experienced	Less-Experienced		
Looking- Forward	3.05	1.228	35 %	43 %	2.96	3.16	0.823	3.21	2.92	0.292
Refactoring-Awareness	3.31	1.111	57 %	30 %	3.54	3.42	0.878	3.54	3.10	0.048*

- 1) **Low level:** refactoring that makes changes only to blocks of code that are completely local and affect elements in the body of a single method only,
- 2) **Medium level:** refactoring that is partially local, but that also significantly changes blocks of code and affects signatures of classes, methods, and fields;
- 3) **High level:** refactoring that changes the signatures of classes, methods, or fields.

Here, the questionnaire focused on two aspects:

- **Planning practice:** Once the need for refactoring is identified, will it then subsequently be planned for in a timely fashion?
- **Execution practice:** Once refactoring has been planned, will it actually be executed in a timely fashion?

To investigate how team characteristics (i.e., quality, change frequency) impact the practice of refactoring, we compared the responses for different groups of similar organizations. The results are shown in Fig. 2. The vertical scale shows the percentage of agreement on both questions. We classified the teams according to the following characteristics:

- **Code Quality:** High-Quality Team vs. Low-Quality team

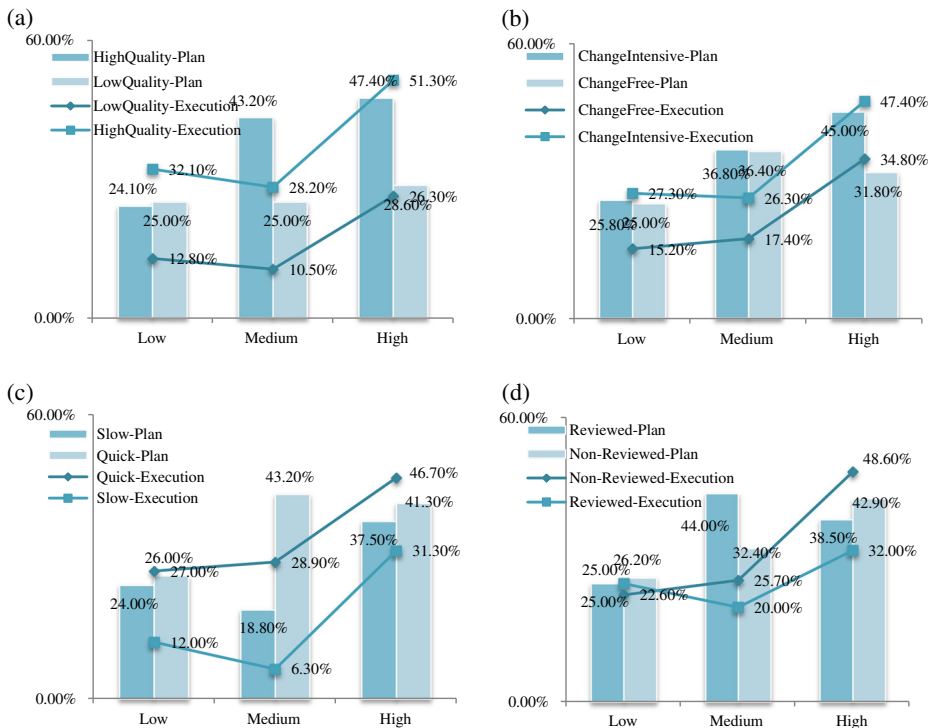


Fig. 2 Practice Difference between two groups

First, we studied whether different refactoring practice would lead to different code quality. We separated the respondents into two groups: Respondents who agreed/strongly agreed that their project still had high quality code structure suitable for supporting extensibility and maintainability were placed in the *High-Quality* team. Respondents who disagreed/strongly disagreed with that position or were neutral with respect to it were placed in the *Low-Quality* team.

Generally, Fig. 2a shows that the *High-Quality* projects usually did efficient planning and execution of refactoring, mostly in a timely manner. For execution, it was clear that *High-Quality* teams prefer to execute all levels of refactoring in a timely manner. For the *Low-Quality* teams, the overall distribution was quite balanced. Only about 25 % of these teams planned for refactoring in a timely manner. However, the refactoring habits of *High-Quality* teams were quite different at different levels. For high level refactoring, approximately half (47.4 %) of the *High-Quality* teams preferred to put them into the plan immediately after they were identified. Some respondents mentioned that low level refactoring was combined with other tasks or that it was not necessary to do it often. It was agreed that it was useful to modify APIs to make them easier for other tasks to use.

- **Frequent Change to Requirements:** Change-intensive environments vs. Change-free environments

Further, we studied how refactoring practice might be affected by the frequency of requirements change by asking the respondents' level of agreement with the statement: "requirements change frequently and the product always deviates from its original design". We divided respondents into two groups: The *Change-intensive* group, which included respondents who either agreed or strongly agreed with the statement, and the *Change-free* group, which included respondents who disagreed, strongly disagreed or remained neutral about the statement.

Figure 2b shows that teams having to deal with more requirements change preferred to execute all levels of refactoring in a very timely manner, and also had a stronger preference for prompt planning of high level refactoring.

- **Iteration Speed:** Slow Team vs. Quick Team

We also compared the impact of iteration length upon refactoring practice. We characterized *Quick* teams are ones for which incremental planning occurred more frequently than once a month, and *Slow* teams, where incremental planning occurred less frequently than once a month.

As shown in Fig. 2c, the *Quick* teams were more likely to plan for medium level refactoring in a timely way, but were less likely to do so for low and high level refactoring. This might be because medium level refactoring is a compromise, replacing the need for frequent low level refactoring while still avoiding the major impacts caused by high level refactoring. In addition, there was no doubt that the *Quick* teams felt that they must execute refactoring plans in a very timely manner.

- **Quality Control:** Reviewed Team vs. Non-Reviewed Team

Finally, we divided respondents into two groups based on their reactions to the statement "the team has enough time to do code/design reviews to ensure code quality". The *Reviewed*

teams were the teams that agreed or strongly agreed with the statement while the *Non-Reviewed* teams comprised respondents who did not.

The results are shown in Fig. 2d. The *Non-Reviewed* teams were more likely to plan for high level refactoring in a timely way, while the *Reviewed* teams were more likely to do timely planning for medium level refactoring. One respondent mentioned that code review and careful design at the beginning of the process should avoid the need for low/medium level refactoring. We also noted that more respondents from *Reviewed* teams favored doing timely low level refactoring, but seemed less supportive of timely execution of high and medium level refactoring.

Summary: It appears that well-executed refactoring plans are a feature that tended to be found in high quality projects. Timely planning for high level refactoring seemed more prevalent in projects that experience more requirements change. Teams whose work is done in relatively shorter iterations seemed more likely to emphasize medium level refactoring. And teams with less time to do the code review plan for the high level refactoring more in time.

4.6 Reasons for Putting off Refactoring

The previous sections revealed that the majority of respondents perceived a lack of concern for refactoring in the development. At the most basic level we are interested in what might cause development teams not to give refactoring a high priority in their planning. To study this, we asked respondents for their reasons, providing ten possible reasons (as shown in Table 10) and also allowing respondents provide additional reasons of their own.

When asked about why teams do not do more extensive refactoring in time, many reasons were cited. Figure 3 shows the percent of respondents who mentioned each of the reasons. The top three reasons for neglecting refactoring during planning were: Reason 1: it can be combined with other tasks, Reason 2: programmers are still able to continue developing and Reason 3: it does not affect overall product performance.

To gain further insights we classified the ten given reasons into four larger groups and tried to match different groups of respondents to each of these different groups of reasons. Specifically, the four groups of reason (denoted by different colors and patterns in Fig. 3) were:

Table 10 Rank of the reasons for different background

	Sketch	Description
Reason 1	Scatter	The refactoring can be combined with other tasks
Reason 2	Workable	Programmers are still able to continue developing
Reason 3:	Product	The postponing of refactoring does not affect overall product performance
Reason 4:	Schedule	Because of tight schedule
Reason 5:	External	Others (project manager or other roles in the team) do not allow to waste time
Reason 6:	Programmer	It is not related to the performance of the programmers
Reason 7:	Complexity	It is simple and requires only a small amount of workload
Reason 8:	Routine	They never plan for refactoring
Reason 9:	Preference	No one wants to do the refactoring
Reason 10:	Others	Others

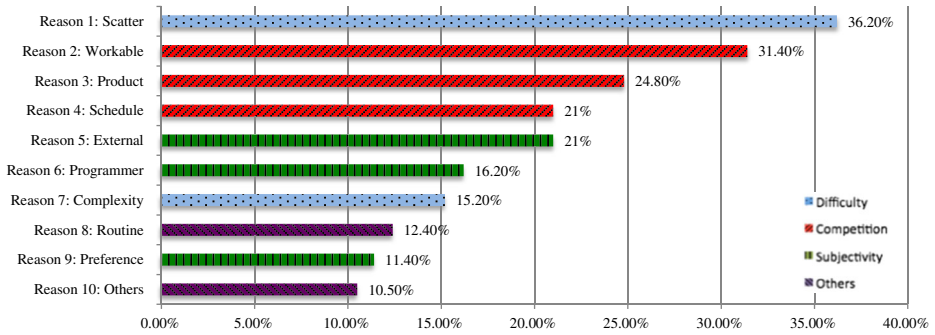


Fig. 3 Potential reasons for putting off refactoring

- **Difficulty of refactoring**

This category includes reasons related to the difficulty of doing refactoring. The category covers both perceptions that refactoring is too easy and that it is too difficult. The reasons that fell into this category were denoted by blue colored bars with points in Fig. 3. They included Reason 1, that refactoring could be combined with other tasks (the most frequently cited reason) and Reason 7, that it is simple and requires only a small amount of workload.

As shown in Fig. 3, many respondents felt that refactoring is so easy that it is not necessary to do it as a standalone task. It was suggested that “simple” refactoring is simply not enough work to be made a unique assignment. Indeed, one respondent noted that developers could do refactoring without difficulty while also doing other tasks. A complication was noted, however, by a respondent who stated that if refactoring were done so unobtrusively then testers might not even notice the need for additional regression testing, thus leading to extra work later on due to the initial neglect of necessary testing.

In contrast, other respondents said refactoring should be put off because it is too “difficult”. Two respondents suggested refactoring might be given a low priority because it was often very hard, especially when the original developer has departed.

- **Competition with other tasks**

This category includes reasons relating to concerns that refactoring comes at the expense of other tasks. Respondents citing these reasons seemed to be aware that refactoring is important, but were worried that the resources to do it would make it harder or impossible to do other things. The reasons cited within this “**Competition**” group are denoted by red colored bars with light upward diagonal pattern in Fig. 3, and include Reasons 2, Reason 3 and Reason 4: because of the tight schedule. One respondent stated that refactoring is not planned for because of tight schedules and heavy workload that never leave any time for doing refactoring. This respondent thought delaying refactoring is necessary and understandable, especially when developing Internet products that need to react to their rapidly changing environment.

On the other hand, some respondents did not appreciate that avoiding refactoring came at a cost because benefits of refactoring such as improved testability, extendibility, and code readability were not clear to these respondents.

- **Subjectivity of developers**

This category includes reasons that reflected subjective judgments made by the developers who performed the refactoring. The reasons included in this “**Subjectivity**” group are denoted by green bars with light vertical pattern in Fig. 3. These reasons include Reason 5: “others (manager or other roles in the project) do not allow to waste time”, Reason 6: “it is not related to the performance of the programmers” and Reason 9: “no one wants to do it”. One respondent observed that upper management would not allocate any time to spend on refactoring. Another three developers (one developer with less than one year of experience, the other two with 1–3 years’ working experience) said that they put off refactoring for no special reason, just because they were never “in the mood”.

Here too it seemed that the benefits of refactoring were not clear to many respondents, causing a lack of passion for it and an unwillingness to advocate for its importance. In a number of other cases, programmers saw little value in refactoring because their performance did not seem to them to be based on the quality of their code, but only upon whether it was “done/not-done” or “how many hours of work are left”. One respondent noted that refactoring is not a sellable feature.

• Others

Finally, there were some reasons that were hard to categorize, and they are indicated by the purple bars with light downward diagonal pattern in Fig. 3. So, for example 12.4 % of the respondents noted that they simply never plan for refactoring (Reason 8). One respondent who is an experienced developer noted that they use points to estimate the overall workload involved in meeting a business requirement but not separately for refactoring and bug fixing tasks. The workload for refactoring and bug fixing are considered parts of the costs of addressing the user story business requirement for code quality.

Among reasons other than those that were predefined (Reason 10), lack of skill and tight schedule were commonly mentioned. Interestingly, there was one respondent who noted that refactoring was allocated a fixed quota of four hours per week regardless of any particular contexts or circumstances.

Being interested in understanding the deeper reasons for these varied attitudes about putting off refactoring, we explored how the reasons varied depending upon the characteristics of the respondents. Accordingly, we investigated differences in the reasons cited by the *Developer* vs. the *Non-Developer* groups. The reasons cited by each of these groups are shown in Table 11, which shows that they mirrored the overall results, Reasons 1 and 2 were the most often cited, and, indeed, the frequency with which most of the other reasons were cited did not vary much between the two groups either. The largest difference between these two groups was found in the case of Reason 4, which is mentioned by 28.3 % of the *Developer* as the third most frequently-mentioned reason, but only 13.5 % of the *Non-Developer* mentioned it, making it the 7th most frequently-mentioned. It is perhaps understandable in that the *Developer* would probably feel schedule tightness far more than *Non-Developer*.

We also looked for differences in opinions and attitudes between *Experienced* and *Less-Experienced* respondents. These results are summarized in right part of Table 11, which is organized in order of the respondents’ experiences. Although, the top three most frequently mentioned reasons were the same for both groups, a significant difference was found in the case of Reason 2. Reason 2 is the most frequently mentioned (by 40.0 %) among the *Less-Experienced* group, but only 22.0 % of the

Table 11 Rank of the reasons for different background

Reason	Non-developer		Developer		P-value	Less-experienced		Experienced		P-value
	Percent	Rank	Percent	Rank		Percent	Rank	Percent	Rank	
Reason 1	38.5 %	1	34.0 %	1	0.633	32.7 %	2	40.0 %	1	0.441
Reason 2	30.8 %	2	32.1 %	2	0.886	40.0 %	1	22.0 %	3	0.048*
Reason 3	25.0 %	3	24.5 %	4	0.956	27.3 %	3	22.0 %	2	0.534
Reason 4	13.5 %	7	28.3 %	3	0.063	21.8 %	5	20.0 %	5	0.820
Reason 5	21.2 %	4	20.8 %	5	0.960	23.6 %	4	18.0 %	6	0.481
Reason 6	17.3 %	6	15.1 %	7	0.759	20.0 %	6	12.0 %	7	0.269
Reason 7	17.3 %	5	13.2 %	8	0.561	10.9 %	8	20.0 %	4	0.198
Reason 8	9.6 %	8	15.1 %	6	0.396	14.5 %	7	10.0 %	9	0.482
Reason 9	9.6 %	9	13.2 %	9	0.565	10.9 %	9	12.0 %	8	0.861

Experienced respondents mentioned it, ranking it as only the third most frequently-mentioned. One reason for this might be that the *Less-Experienced* respondents are simply very focused on continuing to write code that implements new features. Another reason might be inferred from the fact that 20 % of the *Experienced* group cited Reason 7, indicating that they thought that refactoring is simple and requires only a small amount of workload.

5 Post-Survey Interviews

In order to better understand whether there are generalizable rules for refactoring practice, we examined suggestions made by the respondents at the end of the questionnaire and then conducted some interviews. In this section, we summarize our interviews with people having multi-year development experience in multiple domains. Interviewees included technical and management staff members. Parts of these discussions were online and were transcribed for later analysis. We began these discussions with the very general question, “How do you do refactoring in your agile project?” and then let the discussion flow from there.

Comments from two sources were analyzed:

- 1) We received suggestions from 40 respondents and 13 of them participated in further interview. We selected them because they were interested this topic and left their personal information for further contact. 4 of the interviews were done by email and 9 of them were done with live chat tools.
- 2) We started discussion in related discussion groups on LinkedIn. We received comments from 17 group members, of which 3 participated in further interview by email (all of them were scrum coaches).

We got some completely different responses, such as:

- **Continuous refactoring is part of new feature development:**

Some interviewees said that refactoring should be done in conjunction with other feature work. Refactoring is like a technical debt. It is assumed that, in each user story, some refactoring is expected when implementing new features.

These answers mainly came from managers and agile experts. It was suggested that for a team to do continuous refactoring they should be allowed to change the code to fit the new features, but that the team should also find the most elegant and most maintainable way to implement these new features. This was suggested: *“If feature X requires reworking a part of a library somewhere in the software stack the team should do so if they believe this is what will keep the code-base healthy and flexible. This would also be true even if that means reworking parts of the code-base that doesn’t relate directly to the new feature, but would be needed to keep things consistent.”* These were changes that could make the code easier to understand, could make subsequent changes quicker and cheaper, and could also reduce the probability that massive code redesign would be needed. It was suggested that the prospects for long-term success of a product could be improved if bad smells were detected as a new version was developed, and if that new version, containing desired additional functionality, were refactored before being released. These respondents advocated refactoring as the mechanism to be used to keep small messes from getting out of hand, and the mechanism to be used for introducing extensions and changes in a clean, clear, extensible way.

Refactoring is thought of as a part of the development task but not a separate schedulable task. *“The Management and the Customers don’t care about how you do it, but what are you delivering and when. It is the developers’ responsibility.”* One respondent shared that *“In our company, we only estimate workload for the features or some specific technology in the form of story points. But we do not assign refactoring tasks separately as we think it is implied in the feature. Thus, when a story is finished, the related bugs should be fixed and the related refactoring should be done.”* One respondent also thought that refactoring has very little to do with project management and suggested that *“It is a part of the task and as such it is part of any estimate the team provides. I think it isn’t even worth mentioning to any managers involved, but it they do get wind of it and don’t understand I believe they should be able to understand the concept of technical debt and how it is important to avoid accumulating any.”*

- **Do not do refactoring until it is absolutely necessary:**

Others expressed a completely opposite point of view, saying things like, “Don’t do it! If it works, why fix it?” Generally, these respondents seemed to place high priorities on fixing bugs and releasing new features, planning refactoring activities only when either the technical debt becomes very high, (i.e., when code smells are making the system really unmaintainable), or when it is not particularly urgent to add new features or fix bugs.

These kinds of answers were given mostly by developers, especially those who have worked for many years developing complex, mission-critical (and safety-critical) software. *“As source code gets older and/or the number of changes to it increases, understandings of it decrease, and developers seem to become more and more reticent to change code unless they absolutely have to”.* These respondents seemed to assume a sharp distinction, and inherent difference, between tasks for “adding functionality” and tasks for “restructuring the code, keeping the functionality the same”. They seemed to have little confidence in the idea of “Continuous refactoring” (or perhaps “frequent refactoring”) as a way to incrementally improve code structure as new functionality is added, instead viewing it simply as a lot of activity without proportionate reward. These respondents seemed afraid that they would negatively and

seriously affect the functionality (the external behavior) of the code. *“The concept of ‘continuous refactoring’ scares me. When you refactor code, you are changing the code by changing the structure, and regardless of how hard you try, you run the risk of changing the logic also.”* A similar view was provided by another respondent, who said that, *“large sections of code will have to be modified and tested prior to deployment. Management does not necessarily like the idea of that level of testing but it is necessary to rebuild the code base.”*

They were less anxious that the code be very well written and well documented, and that there be clear traceability to specifications. Their attitude seemed to be that if development has been done well, it should be easy to understand what the code is doing and there is then presumably no need for refactoring. *“So why not do things right from the start instead of having to rely on having to make structural changes later?”* In this case, the respondent suggested an emphasis on design *“I call Code Refactoring by the term ‘Design by Code’: instead of analyzing the Software Requirements and determining an elegant, efficient and maintainable method of implementing a solution (which is Design, in any field of engineering), Code Refactoring / Design by Code skips the Design stage and goes straight to Coding. Going straight to Coding by skipping the Design stage can be dangerous because you are programming by trial and error: if you don’t like what you have built (coded), you erase (in effect you tear down) and start again.”* However, this seems to be missing from modern software development practice: *“The problem is that today’s developers are less likely to be software engineers or architects and more likely to be self taught and have a minimal if any formal education. Another problem in today’s environment is that most computerized systems are fast enough to compensate for poorly designed code that is quite bloated with unnecessary redundancy.”*

- **Do the refactoring as planned future work:**

This answer was gathered mostly from respondents developing web products in rapidly changing markets.

Because of the rapid evolution of their markets, these respondents’ systems needed to be refactored every two years (or more often). However, refactoring is not caused in this case by poor design or implementation, but because prior systems have been designed to deal with earlier problems, and have to be upgraded as new problems have been identified subsequently. These respondents suggested *“It is unrealistic to expect that even an excellent designer will not be able to design a system whose structure could remain viable under these conditions”*. These respondents also noted that their agile processes leave only very limited amounts of time for considering architecture. Thus, architects and product managers do not aim for a “perfect” product, but rather a product that will be “satisfactory” given limited time and resources. In this case, the team is quite often aware of which components may need refactoring, why it is necessary to put off the refactoring, and how to deal with known problems. Given all of this, members of these teams expect that code will increasingly need refactoring, but it is then expected that the needed refactoring is likely to be planned for during later stages of development.

- **Various trade-offs:**

Other respondents viewed the need for refactoring as a need that has to be traded off against other needs.

Some suggested budgeting part of each development cycle to refactoring for example, by planning to do separate releases for bug-fixing/refactoring and new features on some sort of alternating basis. *“Basically creating refactoring sprints, to cover basic feature refactoring, and then adding sprints for additional features, then Bugs.”* This might be done by measuring the arrival and fix rates for defects to see if technical debt is increasing or decreasing. Increasing technical debt would then be used to focus work more on bug fixes and refactoring rather than new feature development. One respondent suggested that *“I call most refactoring ‘technical debt.’ My teams create backlog items for technical debt that they as a team have decided are worth the cost. The cost including development, unit testing, QA testing and the potential to introduce new bugs. They need to be able to defend the need for each technical debt item. The team typically includes a few technical debt backlog items in each release in one of the beginning sprints. This allows QA time to adjust their tests and exposes any defects early in the release.”*

Another respondent suggested that there are two kinds of software development: 1) development that creates value or solutions within a short amount of time, and 2) development to create a stub/place holder before there is a clear and complete understanding of what is actually needed. In both of these cases, refactoring was viewed as the act of taking the product and improving it incrementally. Similarly another suggestion was that *“Within my company, I budget 25 % of each development cycle to refactoring. But as a rule, when the developers are introducing new feature-sets to the software platforms, they are tasked with continuous tweaks and improvements as they encounter items that already exist in the code. So as you can see, it is a two pronged approach. For refactoring initiatives, spike stories are played first, and then broken down into much smaller stories for estimation. Once that is done, then the refactoring stories are played as part of the release.”*

6 Implications for Research

These survey results and their analyses suggest the need for a considerable amount of future research. Here are some questions that seem to us to be particularly useful and interesting to pursue.

- Is there some kind of standard strategy that should be followed in planning for refactoring?

Our survey showed that agile teams do careful planning for the tasks of creating new features (98.1 %) and bug fixing (68.6 %) during the planning for iterations in agile software development. But, planning for refactoring is not done nearly as carefully, perhaps because it is not considered particularly difficult or important. This suggests that refactoring should be considered to be a visible task that is well worth careful planning. One direction to pursue is to consider that decisions related to refactoring planning should be made by considering the relationship of refactoring to other kinds of tasks, perhaps guided by metrics such as code quality.

- How do different roles with their different perspectives influence decision-making about the tasks to be performed in the development process? How should they influence this process?

As shown in the study, 46.2 % of the *Non-Developer* thought more than 50 % of the development workload is spent on creating new features in each iteration, while 54.7 % of the *Developer* thought they only spend 31–50 % of their workload to do it. Also, different roles have distinct preferences for planning the different kinds of tasks. Notably, product managers place higher priority on tasks needed to create new features, while project managers and testers place higher priority on bug fixing tasks. Thus, it is necessary to include people who represent a diversity of roles in planning decisions. If decision makers do not have good insight into the benefits of refactoring, it is unrealistic to expect that they will know when to refactor or whether to refactor at all. Also, because of inconsistent awareness of the benefits of refactoring, it seems necessary to provide evaluation criteria that can be used to justify the expenditure of resources required for refactoring. In carrying out planning, we suggest considering questions such as: “How to objectively distinguish between a well-thought-out policy decision and a decision that is just shortsighted?”, and “How can a team be sure that all relevant roles and perspectives are appropriately represented in group decision-making about the priorities to be assigned to the tasks to be scheduled?”

- Is it necessary to increase the priority of both types of refactoring in iteration planning?

The study has indicated that many respondents have been expecting to see higher priorities placed upon both kinds of refactoring. Indeed, product managers and testers wanted to see a reduction in the priority placed on creating new features. Moreover, the survey shows that, although higher quality projects prefer to both plan for and execute refactoring in a timely manner, refactoring is all too often put off because of compromises. It seems clear that the priority placed on refactoring should be based upon quantitative measures of such things as cost, quality, and return on investment. These measures should be agreed upon and taken as a way as to guide prioritization discussions and decisions. We suggest considering questions such as: “How to objectively determine when delaying refactoring is indicated?”, and “What quantitative advantages can be expected to accrue from specific kinds of refactoring in different project contexts?”

- How can deferring floss refactoring be shown to impact development process?

Our survey results show that deferred floss refactoring is considered to have four negative effects that were enumerated above, but also that frequent floss refactoring can help avoid root-canal refactoring. Notably, 79.6 % of the respondents agreed that deferring floss refactoring would necessitate a lot of extra effort to do the refactoring needed to clean up code later on. However, some of the refactoring is put off for subjective reasons, such as those mentioned by developers, as described in the previous section of this paper. In planning, we suggest considering “What incentives (such as performance evaluation points or weights) are effective in causing developers to be more objective in assessing the desirability of the inclusion of more refactoring tasks in project planning?”

- How do different levels of experience influence the practice of refactoring during development?

As shown in the study, the top reason that the *Experienced* group cited for putting off refactoring is that it could be done together with other tasks. However, the *Non-Experienced*

group was focused primarily upon being able to continue development. Also, the more *Experienced* group's focus on extensibility and maintainability to expedite future changes caused this group to become aware of the need for refactoring earlier. This suggests that awareness of refactoring should become a skill to be developed from experience. Thus, we suggest that it is could be particularly important to study how the building of this skill might be expedited, perhaps through creation of some kind of refactoring guide to be mandated for use by the less experienced members of a development team.

7 Threats to Validity

The research outcomes presented in this paper must be viewed within bounds created by some limitations that we address next:

External validity: The main threat to the external validity of our survey is that it may not have attracted a sufficiently large, diverse, and representative sample of the population, which is generally difficult to obtain for studies in software engineering. The methodology of asking a few people for their opinions without a mediated focus group has its limitations, even if the questions are tested in advance. It is also possible that bias and other psychological factors may have influenced overall research results. Although we do not claim to have taken a truly random sample from a defined population, our exclusion criteria at least helped us to screen out many inappropriate individuals.

Internal validity: We targeted individuals but asked them about team information and company issues. The results are naturally bound to be affected by subjective opinions, attitudes and current knowledge of the individual respondents, and may thus represent a narrow view of each organization as a whole. Also, responses from the same team or company might well not be independent. It was indeed not possible to identify how many of the respondents belonged to a particular company, project or team.

Construct validity: The survey was conducted anonymously, and we did not collect further information on the business domain of the respondents. Different backgrounds, knowledge and experiences of the respondents are also likely to affect their understanding and interpretations of planning strategy and the different kinds of refactoring mentioned in the questionnaire. To reduce misleading responses due to respondents' lack of knowledge, the 'I do not know' alternative was provided.

8 Conclusions and Future Work

The goal of this study was to provide insights into the practice of refactoring in agile software development processes. This questionnaire-based survey research provided results from 105 respondents.

The principal results concerning iteration planning in agile software development processes are as follows: during iteration planning 98.1 % of agile teams do careful planning for the tasks of creating new features but only around 40 % of the teams do similarly careful planning for refactoring. The top three reasons for neglecting refactoring during planning are that: it should be combined with other tasks, it does

not prevent programmers from continuing with new feature development, and it does not affect product performance. Our results show significant evidence that different roles attach distinctly different priorities to the different kinds of tasks in agile development. The priority of the tasks involved in creating new features is higher when product managers have higher authority in planning.

Respondents also told us what they think about the workload and the priority of each of the different kinds of task. 46.2 % of *Non-Developer* thought that the tasks of creating new features take more than 50 % of the workload during each iteration, while only 22.6 % of the *Developer* thought so. Our results provide significant evidence that respondents would like the priority assigned to refactoring tasks to be raised. We found that 79.6 % of the respondents agreed that deferring floss refactoring will necessitate a considerable amount of extra effort to clean up the code later on, and that frequent floss refactoring can be helpful in avoiding this so-called root-canal refactoring. Our results suggest that good quality in a project seems at least partially attributable to the existence of well-executed plans. There is a distinct tendency to emphasize high level refactoring on projects that are subjected to frequent changes in requirements.

The results of our study contribute to our understanding of how software development teams do planning and practice refactoring. We intend to follow up this work as follows:

- 1) **Process Model.** We will create process models for agile software development projects to compare different approaches to doing agile development and also to compare agile development to some non-agile approaches. The characteristics and statistics collected and inferred from this survey will enable us to define the details of these processes more accurately.
- 2) **Iteration Planning Strategy.** We will identify areas in which to develop tools for supporting the improvement of scheduling and planning refactoring tasks in software development.

Acknowledgments This research was supported by the Natural Science Foundation of China under grant Nos. 91318301, 91218302, 61432001. This work was also supported by the US National Science Foundation under Awards No. CCR-0205575, CCR-0427071, and IIS-0705772.

APPENDIX-Questionnaire

Part A. Background Information

(Options in this part refer Table 1 in section 4.1.)

1. Over how many years have you been involved in Software Engineering?
2. How many employees in your organization?
3. How many years of experience do you have working in Agile methodologies?
4. What is your roles in the team? (Multiple choice)
5. Where is the team located?
6. What is the iteration length in the team?
7. How many employees in the team?
8. Who is the product owner?

Part B. Planning Practice

9. What proportion of workload does a development engineer spend for each kind of task? (<1 %, 1–10 %, 11–30 %, 31–50 %, >50 %)
 - A. Implementing new features
 - B. Bug fixing
 - C. Floss Refactoring
 - D. Root-canal Refactoring
10. How do the following roles take part in the iteration planning? (Min number: 0-Max number: 5, I don't know)
 - A. Project Manager
 - B. Product Manager
 - C. Development Engineer
 - D. Test Engineer
11. What is the priority of the four kinds of tasks when making an iteration plan? (Min number: 0-Max number: 5, I don't know)
12. How does the team plan for the four kinds of tasks of new features?
 - A. Do not make a plan to accomplish a task, but leave a needed task to be done when someone has spare time;
 - B. Do not make a plan to accomplish a task, but leave some slack time to allow for the task to be done;
 - C. Plan that a needed task will be done as part of another task;
 - D. Make a simple plan for the critical tasks;
 - E. Make a specific detailed plan for all tasks.
 - F. I don't know,
 - G. Others.

Part C. Refactoring Practice

Please indicate your level of agreement of the following statements based on your experience of agile software development practices in your team. (Strongly Disagree, Disagree, Neither Agree nor Disagree, Agree, Strong Agree)

13. In the project, requirements change frequently and the product always deviates from its original design.
14. In the project, the team has enough time to do code/design reviews to ensure the code quality.
15. When development engineers create new features, they consider the extendibility and maintainability for most possible future changes.
16. In the project, the development engineers usually identify the necessity of floss refactoring before it must be done (the product has poor performance/blocks new changes).
17. In the project, the deferred floss refactoring will largely reduce the productivity in adding new features in the related components.

18. In the project, the deferred floss refactoring will largely increase the workload of adding new features in the related components.
19. In the project, the deferred floss refactoring will largely increase the defect density of adding new features in the related components.
20. In the project, the deferred floss refactoring will require a lot of extra effort to clean up the code later.
21. In the project, root-canal refactoring can be avoided by frequent floss refactoring.
22. In the project, once the need for refactoring is identified, the refactoring will subsequently be planned (or clearly plan to combine it with some new feature implement) for in a timely fashion.
 - A. Low-level
 - B. Medium-level
 - C. High-level
23. In the project, once refactoring has been planned, it will actually be executed in a timely fashion.
 - A. Low-level
 - B. Medium-level
 - C. High-level
24. Now, the project still have high quality of code structure suitable for supporting extendibility and maintainability.

Part D. Personal Opinion

25. In real project, why team doesn't put the floss refactoring into the plan in time?
26. What is your ideal priority of the following tasks for the programmer? (Min number: 0- Max number: 5, I don't care)
27. Any suggestion?

References

- Alshayeb M, Li W (2005) An empirical study of system design instability metric and design evolution in an agile software process. *J Syst Softw* 74(3):269–274
- Beck K (2000) *Extreme programming explained: embrace change*. Addison-Wesley Professional
- Begel A, Nagappan N (2007, September). Usage and perceptions of agile software development in an industrial context: An exploratory study. In *Empirical Software Engineering and Measurement, 2007. ESEM 2007. First International Symposium on* (pp 255–264). IEEE
- Belady LA, Lehman MM (1976) A model of large program development. *IBM Syst J* 15(3):225–252
- Bellomo S, Nord RL, Ozkaya I (2013, May). A study of enabling factors for rapid fielding combined practices to balance speed and stability. In *Software Engineering (ICSE), 2013 35th International Conference on* (pp 982–991). IEEE
- Brasil MMA, da Silva TGN, de Freitas FG, de Souza JT, Cortés MI (2012) A multiobjective optimization approach to the software release planning with undefined number of releases and interdependent requirements. In *Enterprise Information Systems* (pp 300–314). Springer Berlin Heidelberg
- Buchmann F, Nord RL, Ozakaya I (2012) *Architectural Tactics to support rapid and agile stability*. Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst

- Cao L, Ramesh B, Abdel-Hamid T (2010) Modeling dynamics in agile software development. *ACM Trans Manag Inf Syst (TMIS)* 1(1):5
- Chen J, Xiao J, Wang Q, Osterweil LJ, Li M (2014, May). Refactoring planning and practice in agile software development: an empirical study. In *Proceedings of the 2014 International Conference on Software and System Process* (pp 55–64). ACM. (Best paper award)
- Chen L, Babar MA (2014) “Towards an Evidence-Based Understanding of Emergence of Architecture through Continuous Refactoring in Agile Software Development.” *Software Architecture (WICSA), 2014 IEEE/IFIP Conference on*. IEEE
- Dig D, Johnson R (2005, September). The role of refactorings in API evolution. In *Software Maintenance, 2005. ICSM’05. Proceedings of the 21st IEEE International Conference on* (pp 389–398). IEEE
- Dong X, Yang QS, Wang Q, Zhai J, Ruhe G (2011, December). Value-Risk Trade-off Analysis for Iteration Planning in eXtreme Programming. In *Software Engineering Conference (APSEC), 2011 18th Asia Pacific* (pp 397–404). IEEE
- Drury M, Conboy K, Power K (2012) Obstacles to decision making in agile software development teams. *J Syst Softw* 85(6):1239–1254
- Dybå T, Dingsøyr T (2008) Empirical studies of agile software development: a systematic review. *Inf Softw Technol* 50(9):833–859
- Elssamadisy A, Schalliol G (2002, May). Recognizing and responding to bad smells in extreme programming. In *Proceedings of the 24th International conference on Software Engineering* (pp 617–622). ACM
- Fink A (Ed) (2003) *The survey handbook* (Vol. 1). Sage
- Fowler M (1999) *Refactoring: improving the design of existing code*. Pearson Education India
- Harman M, Tratt L (2007, July) Pareto optimal search based refactoring at the design level. In *Proceedings of the 9th annual conference on Genetic and evolutionary computation* (pp 1106–1113). ACM
- Jones JA, Harrold MJ, Stasko J (2002, May) Visualization of test information to assist fault localization. In *Proceedings of the 24th international conference on Software engineering* (pp 467–477). ACM
- Kataoka Y, Imai T, Andou H, Fukaya T (2002) A quantitative evaluation of maintainability enhancement by refactoring. In *Software Maintenance, 2002. Proceedings. International Conference on* (pp 576–585). IEEE
- Kim M, Zimmermann T, Nagappan N (2012, November) A field study of refactoring challenges and benefits. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*(p. 50). ACM
- Kolb R, Muthig D, Patzke T, Yamauchi K (2006) Refactoring a legacy component for reuse in a software product line: a case study. *J Softw Maint Evol Res Pract* 18(2):109–132
- Leitch R, Stroulia E (2003, May) Understanding the economics of refactoring. In *EDSER-5 5 th International Workshop on Economic-Driven Software Engineering Research* (p. 44)
- Marinescu R (2005, September) Measurement and quality in object-oriented design. In *Software Maintenance, 2005. ICSM’05. Proceedings of the 21st IEEE International Conference on* (pp 701–704). IEEE
- Moser R, Abrahamsson P, Pedrycz W, Sillitti A, Succi G (2008) A case study on the impact of refactoring on quality and productivity in an agile team. In *Balancing Agility and Formalism in Software Engineering* (pp 252–266). Springer Berlin Heidelberg
- Moser R, Sillitti A, Abrahamsson P, Succi G (2006) Does refactoring improve reusability?. In *Reuse of Off-the-Shelf Components* (pp 287–297). Springer Berlin Heidelberg
- Murphy-Hill E, Pamin C, Black AP (2012) How we refactor, and how we know it. *Softw Eng IEEE Trans* 38(1):5–18
- Negara S, Chen N, Vakilian M, Johnson RE, Dig D (2012) Using Continuous Code Change Analysis to Understand the Practice of Refactoring
- Nidhra S, Dondeti J, Katikar P, Tekkali S (2012, September) Implementing the concept of refactoring in software development. In *Software Engineering (CONSEG), 2012 CSI Sixth International Conference on* (pp 1–8). IEEE
- Przepiora M, Karimpour R, Ruhe G (2012, September) A hybrid release planning method and its empirical justification. In *Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement* (pp 115–118). ACM
- Rachatasumrit N, Kim M (2012, September) An empirical investigation into the impact of refactoring on regression testing. In *Software Maintenance (ICSM), 2012 28th IEEE International Conference on* (pp 357–366). IEEE
- Ratzinger J, Sigmund T, Gall HC (2008, May) On the relation of refactorings and software defect prediction. In *Proceedings of the 2008 international working conference on Mining software repositories* (pp 35–38). ACM
- Reddy KR, Rao AA (2009) Dependency oriented complexity metrics to detect rippling related design defects. *ACM SIGSOFT Softw Eng Notes* 34(4):1–7
- Sahraoui HA, Godin R, Miceli T (2000) Can metrics help to bridge the gap between the improvement of oo design quality and its automation?. In *Software Maintenance, 2000. Proceedings. International Conference on* (pp 154–162). IEEE
- Schwaber K, Beedle M (2002) *Agile Software Development with Scrum*

- Schwaber K, Sutherland J (2011). Scrum guidebook. Scrum. org and Scrum Inc
- Simon F, Steinbruckner F, Lewerentz C (2001). Metrics based refactoring. In Software Maintenance and Reengineering, 2001. Fifth European Conference on (pp 30–38). IEEE
- Soares G, Gheyi R, Serey D, Massoni T (2010) Making program refactoring safer. *Software IEEE* 27(4):52–57
- Venables WN, Smith DM, R Development Core Team (2002) An introduction to R
- Xing Z, Stroulia E (2005, November) UMLDiff: an algorithm for object-oriented design differencing. In Proceedings of the 20th IEEE/ACM international Conference on Automated software engineering (pp 54–65). ACM
- Yamashita A, Moonen L (2013, October) Do developers care about code smells? An exploratory survey. In Reverse Engineering (WCRE), 2013 20th Working Conference on (pp 242–251). IEEE
- Zhang L, Kim M, Khurshid S (2011, September) Localizing failure-inducing program edits based on spectrum information. In Software Maintenance (ICSM), 2011 27th IEEE International Conference on (pp 23–32). IEEE



Jie Chen is currently working toward the graduate degree at the Lab of Internet Software Technologies, Institute of Software Chinese Academy of Sciences. She received the BS degree in software engineering from Xiamen University in 2009, also a BS degree (secondary one) in Economics from Department of Economics at the same university. Her research interests include resource scheduling, process analysis and simulation. She is currently doing research to refactoring practice in the process.



Junchao Xiao is an Associate Professor in the Institute of Software at Chinese Academy of Sciences (ISCAS). He got his Ph.D. in Computer Science from ISCAS in 2007. He was a visiting scholar in the Department of Computer Science, University of Massachusetts Amherst from November 2008 to November 2009. His research

interests include software process modeling, dynamic resource scheduling in complex processes and systems. He has published more than twenty papers in journals and conferences.



Qing Wang is a Professor of the Institute of Software, Chinese Academy of Sciences. She is the Director of the Lab for Internet Software Technologies. She has intensive research and industry experience in the area of software process, including software process technologies and quality assurance and requirement engineering. She is a member of Cloud Computing Experts Association under Chinese Institute of Electronics and a Lead Appraiser of SEI CMMI SCAMPI. Qing Wang has led and is leading many important domestic and international cooperative projects. She has won various kinds of awards such as the National Award for Science and Technology Progress. She also is general co-chair of ESEIW 2015, PC member of some academy conferences and program co-chair of SPW/ProSim 2006, ICSP 2007 and ICSP 2008.



Leon J. Osterweil is a Professor in the College of Information and Computer Science at University of Massachusetts Amherst, where he had previous served as Dean of the College of Natural Sciences and Mathematics. He is a Fellow of the ACM. His paper suggesting the idea of process programming was recognized as the Most Influential Paper of the 9th International Conference on Software Engineering (ICSE 9), awarded as a 10-year retrospective. Prof. Osterweil was the recipient of the SIGSOFT Outstanding Research Award (2003), the SIGSOFT Influential Educator Award (2010), and the SIGSOFT Distinguished Service Award (2014). Prof. Osterweil has been the program chair and general chair of many Software Engineering conferences including

serving as the General Chair of the 28th International Conference on Software Engineering (ICSE 28). He is a director of the International Software Process Association.



Mingshu Li is a Professor of the Institute of Software, Chinese Academy of Sciences. He received a Ph.D degree from the Department of Computer Science, Harbin Institute of Technology in 1993; also a Master degree (secondary one) in Economics from Department of Social Sciences at the same university in 1995. He took his post-doctoral research in Institute of Software at the Chinese Academy of Sciences during 1993–1995 and Department of Artificial Intelligence at University of Edinburgh during 1995–1996, respectively. He is a Fellow of China Computer Federation (CCF). His major research interests are software process, software engineering methodology, requirements engineering, and operating systems. He has published and co-authored more than one hundred research papers.